

Guide d'hébergement

De l'ordinateur de développement au site en ligne sur paradigital.tech — Render, Supabase, GitHub et LWS. Situation au 27 septembre 2026.

Le site de l'agence est en ligne sur **<https://paradigital.tech>**. Il repose sur quatre services qui ont chacun un rôle précis : **GitHub** garde le code, **Render** fait tourner le site et l'API, **Supabase** héberge la base de données, et **LWS** gère le nom de domaine et les boîtes mail. Ce guide retrace le chemin parcouru, étape par étape, en expliquant à chaque fois pourquoi on l'a fait — pour pouvoir le refaire, le modifier ou le dépanner seul.

Secrets

Aucun mot de passe ni clé secrète ne figure dans ce document. Les valeurs sensibles sont remplacées par `<MOT_DE_PASSE>` ou `<réf-projet>` ; les vraies valeurs se trouvent dans `backend/.env` (jamais envoyé sur GitHub) et dans l'onglet **Environnement** de Render.

Vue d'ensemble

Le schéma ci-dessous montre qui parle à qui quand un visiteur ouvre le site. Les flèches pleines sont les échanges normaux ; en pointillés bleus, l'envoi des formulaires ; en pointillés rouges, ce qui ne fonctionne pas encore.

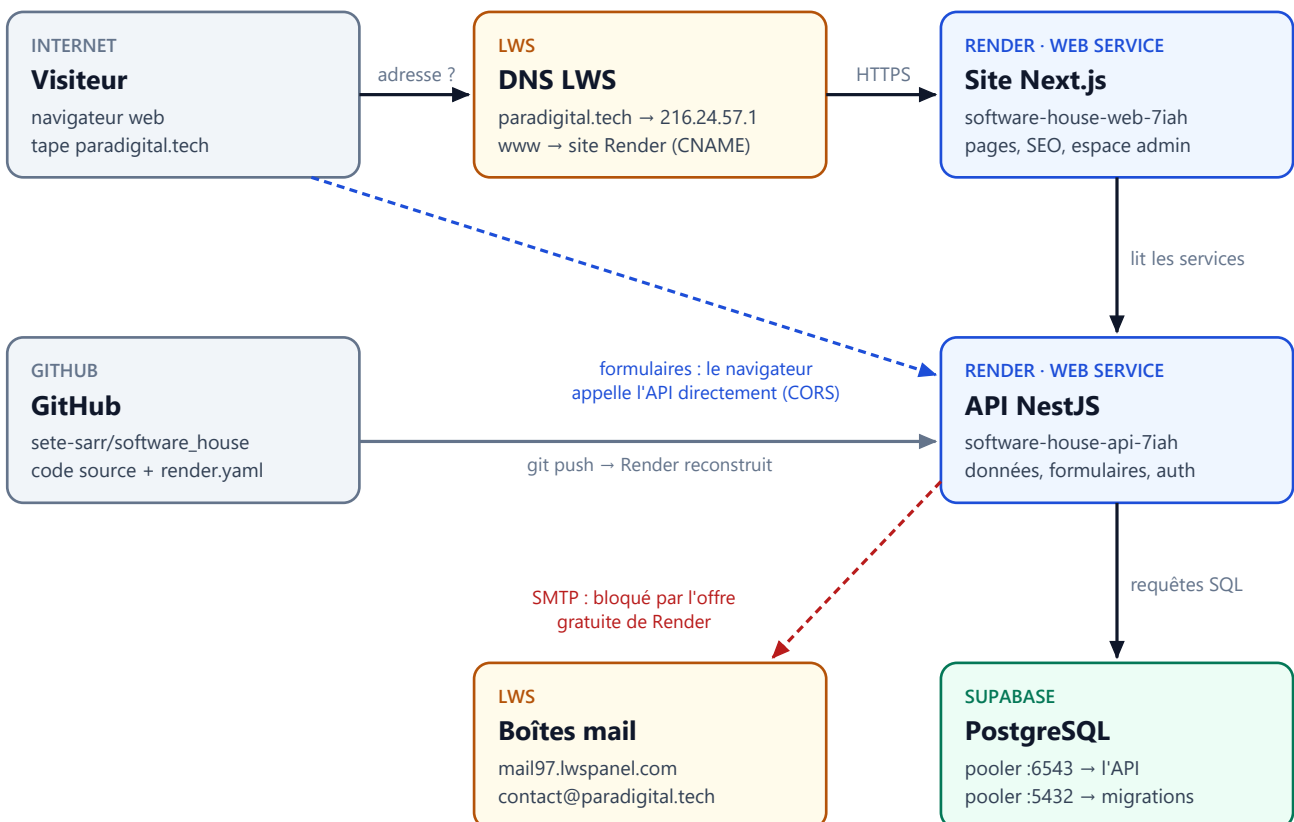


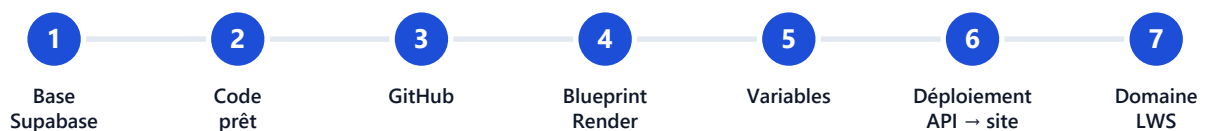
Figure 1 — Architecture en production. Le site et l'API sont deux services Render distincts ; seule l'API a accès à la base de données.

Les quatre acteurs

Service	Rôle	Ce qu'on y a configuré	Coût actuel
GitHub	Stocke le code source. Chaque envoi (push) déclenche un nouveau déploiement.	Dépôt <code>sete-sarr/software_house</code> , branche <code>main</code> .	Gratuit
Render	Construit et fait tourner le site (Next.js) et l'API (NestJS), fournit le HTTPS.	2 services via <code>render.yaml</code> , région Frankfurt, variables d'environnement, domaine personnalisé.	Gratuit (plan Free)
Supabase	Base de données PostgreSQL managée (sauvegardes, sécurité).	Tables créées par les migrations Prisma, 11 services et un compte admin insérés.	Gratuit
LWS	Nom de domaine <code>paradigital.tech</code> et boîtes mail.	Zone DNS : le domaine pointe maintenant vers Render. MX et mail inchangés.	Déjà payé (domaine)

Le parcours en 7 étapes

L'ordre compte : chaque étape s'appuie sur la précédente. La base de données doit exister avant l'API, l'API doit tourner avant la construction du site, et le site doit être en ligne avant de lui donner votre nom de domaine.



Étape 1 — Créer la base de données sur Supabase

En développement, PostgreSQL tournait dans Docker sur votre ordinateur. En production, il faut une base accessible depuis Internet, sauvegardée et sécurisée : c'est le rôle de Supabase.

Point clé : deux adresses de connexion. Supabase propose un pooler (un intermédiaire qui partage les connexions) sous deux modes, et chacun a son usage :

Variable	Port	Mode	Utilisée par
DATABASE_URL	6543	Transaction : idéal pour beaucoup de requêtes courtes	L'API NestJS, à chaque requête d'un visiteur
DIRECT_URL	5432	Session : garde la connexion ouverte	Prisma pour les migrations (<code>prisma migrate deploy</code>), qui ne fonctionnent pas en mode transaction

Format des adresses (le mot de passe doit être encodé s'il contient `@` : `/` `#` `?` `%`) :

```
DATABASE_URL="postgresql://postgres.<réf-projet>:<MOT_DE_PASSE>@aws-1-eu-west-1.pooler.supabase.com:6543/postgres?uselibpqcompat=true&sslmode=require"
DIRECT_URL="postgresql://postgres.<réf-projet>:<MOT_DE_PASSE>@aws-1-eu-west-1.pooler.supabase.com:5432/postgres?uselibpqcompat=true&sslmode=require"
```

Le fichier `backend/prisma.config.ts` utilise `DIRECT_URL` s'il existe, sinon `DATABASE_URL` — ainsi la base locale Docker continue de fonctionner sans rien changer. On a ensuite créé les tables et les données de départ :

```
cd backend
npx prisma migrate deploy      # crée les tables (2 migrations)
npx prisma db seed             # 11 services + 1 compte admin
```

À savoir

`sslmode=require` chiffre la connexion, mais sans vérifier le certificat de Supabase (qui utilise sa propre autorité). Pour une sécurité maximale, télécharger le certificat CA dans Supabase → Project Settings → Database → SSL Configuration et passer en `sslmode=verify-full`.

Étape 2 — Préparer le code pour la production

Avant d'envoyer le code, plusieurs ajustements ont été nécessaires. Aucun n'est propre à Render seul : ce sont des bonnes pratiques pour tout hébergement derrière un proxy.

Modification	Pourquoi
Fichier <code>render.yaml</code> (Blueprint)	Décrit les 2 services, leurs commandes de build/démarrage et leurs variables. L'infrastructure est versionnée avec le code et recréable en quelques clics.
<code>TRUST_PROXY=1</code> dans l'API	Sur Render, chaque requête passe par un proxy. Sans ce réglage, l'API voit l'IP du proxy : la limite anti-spam serait partagée par tous les visiteurs.
Migrations au démarrage	Le plan gratuit n'a pas d'étape « pre-deploy » : la commande de démarrage lance <code>prisma migrate deploy</code> puis l'API. Sans danger : une migration déjà appliquée est ignorée.
Fichiers <code>package-lock.json</code> régénérés avec <code>npm 10</code>	Render utilise <code>npm 10</code> (fourni avec Node 22) et votre PC <code>npm 11</code> : les deux n'écrivent pas le fichier de la même façon (voir « Problèmes rencontrés »).
<code>CORS_ORIGIN</code> accepte plusieurs adresses	Pendant le passage au nouveau domaine, l'ancienne et la nouvelle adresse du site doivent pouvoir envoyer des formulaires. Les espaces et le « / » final sont ignorés.
E-mails envoyés en arrière-plan	Le visiteur reçoit sa confirmation dès que sa demande est enregistrée, sans attendre le serveur mail. Un envoi bloqué abandonne au bout de 10 à 20 secondes.
SMTP sur le port 465	Le port 465 exige une connexion chiffrée dès le départ ; le code le détecte automatiquement.

Étape 3 — Envoyer le code sur GitHub

Render ne lit pas votre ordinateur : il récupère le code sur GitHub. Chaque modification suit donc le même trajet :

```
git add .
git commit -m "Description de la modification"
git push
```

Les fichiers `.env` (mots de passe) sont exclus par `.gitignore` et ne partent jamais sur GitHub. Seuls les fichiers modèles `.env.example`, sans secret, sont envoyés.

Étape 4 — Créer les services avec le Blueprint Render

Un **Blueprint** est un plan de construction : Render lit `render.yaml` et crée tout seul les services décrits dedans.

1. Sur **render.com**, se connecter avec son compte GitHub.

2. **New** → **Blueprint**, puis choisir le dépôt **software_house**.
3. Render affiche les 2 services (**software-house-api** et **software-house-web**) et demande les variables marquées « à renseigner ». Les secrets JWT sont générés automatiquement.
4. **Apply** : Render construit et démarre les deux services.

Suffixe ajouté par Render

Les noms simples étaient déjà pris par d'autres utilisateurs : Render a ajouté le suffixe **-7iah**. Les adresses réelles sont donc **software-house-web-7iah.onrender.com** et **software-house-api-7iah.onrender.com**. Toujours vérifier l'adresse affichée par Render avant de la recopier dans une variable.

Étape 5 — Renseigner les variables d'environnement

Les variables sont les réglages qui changent entre votre PC et la production (adresses, mots de passe). Elles se gèrent dans chaque service Render, onglet **Environment**.

Service	Variable	Valeur	Remarque
API	DATABASE_URL	Adresse Supabase, port 6543	Secret
API	DIRECT_URL	Adresse Supabase, port 5432	Secret
API	CORS_ORIGIN	https://paradigital.tech , https://software-house-web-7iah.onrender.com	Adresses autorisées à envoyer des formulaires
API	JWT_SECRET, JWT_REFRESH_SECRET	Générées par Render	Ne pas modifier
API	SMTP_HOST / SMTP_PORT	mail97.lwspanel.com / 465	Voir « Reste à faire » (e-mails)
API	SMTP_USER / SMTP_PASSWORD	Adresse complète et mot de passe de la boîte mail	Secret
Site	NEXT_PUBLIC_API_URL	https://software-house-api-7iah.onrender.com/api/v1	Intégrée au moment du build
Site	NEXT_PUBLIC_SITE_URL	https://paradigital.tech	Adresse officielle (SEO, sitemap)

Piège fréquent

Coller **uniquement la valeur** dans le champ, jamais « NOM=valeur » ni le nom suivi d'une tabulation. Deux déploiements ont échoué à cause d'un tel copier-coller. Après modification d'une variable **NEXT_PUBLIC_...**, choisir **Save, rebuild, and deploy** : ces valeurs sont figées dans le site au moment de sa construction.

Étape 6 — Déployer dans le bon ordre : l'API, puis le site

Pendant sa construction, le site génère à l'avance certaines pages (par exemple **/services**) en interrogeant l'API. Si l'API n'est pas en ligne à ce moment-là, la construction échoue avec « Impossible de charger les services ».

1. Vérifier que l'API est **Deployed** et que <https://software-house-api-7iah.onrender.com/api/v1/health> affiche `"database":{"status":"up"}`.
2. Ensuite seulement, lancer (ou relancer) le déploiement du site : **Manual Deploy** → **Deploy latest commit**.

En cas d'échec, Render conserve l'ancienne version en ligne : un déploiement raté ne coupe pas le site.

Étape 7 — Afficher le domaine paradigital.tech

Deux endroits à configurer : Render doit savoir qu'il répond pour paradigital.tech, et LWS doit envoyer les visiteurs vers Render. C'est le rôle du DNS, l'« annuaire » d'Internet qui traduit un nom en adresse.

Dans Render — service du site → **Settings** → **Custom Domains** → **Add** : paradigital.tech. Render ajoute www.paradigital.tech automatiquement.

Dans LWS — espace client → Domaines → paradigital.tech → **Zone DNS** :

Enregistrement	Avant	Après	Effet
A @	213.255.195.58 (hébergement LWS)	216.24.57.1	paradigital.tech → Render
AAAA @	Adresse IPv6 LWS	Supprimé (automatiquement)	Render ne gère pas l'IPv6
CNAME www	paradigital.tech	software-house-web-7iah.onrender.com	www → site Render
MX, A mail, TXT (SPF)	—	Inchangés	Vos e-mails continuent de fonctionner

Il suffit ensuite de patienter (de quelques minutes à quelques heures), puis de cliquer sur **Verify** dans Render. Render obtient alors gratuitement un certificat HTTPS (« Certificate Issued ») et redirige automatiquement http:// vers https://.

Problèmes rencontrés et solutions

Chaque problème ci-dessous s'est réellement produit pendant la mise en ligne. Garder ce tableau sous la main : ce sont les pannes les plus probables si vous refaites un déploiement.

Symptôme	Cause	Solution
git add : « 'backend/' does not have a commit checked out »	backend/ et frontend/ contenaient chacun leur propre dépôt Git, créé par les outils d'initialisation.	Dossiers .git imbriqués retirés ; un seul dépôt à la racine.
Page /services en local : « Impossible de charger les services »	Docker Desktop arrêté, donc pas de base de données.	Démarrer Docker, lancer PostgreSQL, appliquer les migrations.
Supabase : « password authentication failed »	L'adresse contenait encore [YOUR-PASSWORD], puis la réinitialisation n'avait pas été validée.	Réinitialiser le mot de passe dans Supabase (bouton de confirmation) et l'encoder dans l'adresse.
Render : npm ci « Missing: @emnapi/core from lock file »	package-lock.json écrit par npm 11 (PC), lu par npm 10 (Render).	Régénérer avec npx npm@10 install --package-lock-only.
Build du site : « Invalid URL » avec EXT_PUBLIC_SITE_URL dans la valeur	Le nom de la variable et une tabulation collés dans le champ valeur.	Effacer le champ et ne coller que l'adresse.
Build du site : « Impossible de charger les services »	Adresse d'API sans le suffixe -7iah : le site appelait un service inexistant.	Corriger NEXT_PUBLIC_API_URL, puis reconstruire le site.
API : « CORS_ORIGIN must be a URL address »	Même erreur de copier-coller dans la valeur.	Ne coller que l'adresse ; l'API tolère désormais espaces et « / » final.

Symptôme	Cause	Solution
Formulaire de contact : plus de 2 minutes d'attente	L'API attendait l'envoi des e-mails, et Render (gratuit) bloque le SMTP.	E-mails envoyés en arrière-plan avec délai maximal ; la demande est enregistrée immédiatement.
SMTP : « Hostname does not match certificate »	Le certificat du serveur mail est au nom de *.lwspanel.com.	Utiliser le vrai serveur : mail197.lwspanel.com.
Render : « Certificate error » après « Verified »	Les nouvelles valeurs DNS n'étaient pas encore visibles partout.	Attendre la propagation, puis Verify à nouveau.

Au quotidien

Je veux...	Je fais...
Mettre en ligne une modification du code	<code>git add . → git commit -m "..." → git push</code> . Render reconstruit tout seul le service concerné.
Changer un réglage (adresse, mot de passe)	Render → service → Environment → modifier → Save and deploy (Save, rebuild, and deploy) pour les variables NEXT_PUBLIC du site).
Comprendre une panne	Render → service → onglet Logs . Les 30 dernières lignes suffisent en général.
Vérifier que l'API et la base vont bien	<code>https://software-house-api-7iah.onrender.com/api/v1/health</code>
Ajouter une table ou un champ en base	En local : <code>npx prisma migrate dev --name <nom></code> , puis push : Render applique la migration au démarrage.
Installer un paquet npm	<code>npx npm@10 install <paquet></code> pour garder un <code>package-lock.json</code> compatible avec Render.

Mise en veille

Sur le plan gratuit, un service sans visite depuis 15 minutes s'endort. Le visiteur suivant attend environ une minute le temps du réveil. C'est normal, pas une panne.

Ce qui reste à faire

Le site est en ligne et fonctionnel. Voici les points ouverts, du plus urgent au moins urgent.

Priorité	Action	Pourquoi
Haute	Mettre NEXT_PUBLIC_SITE_URL = <code>https://paradigital.tech</code> et mettre à jour CORS_ORIGIN, puis reconstruire le site.	Sinon Google référence l'adresse Render au lieu de paradigital.tech.
Haute	Vérifier le certificat de <code>www.paradigital.tech</code> dans Render (Verify).	www ne répondait pas encore en HTTPS lors de la dernière vérification.
Haute	Choisir une solution pour les e-mails : plan Starter Render pour l'API (≈ 7 \$/mois, garde LWS) ou service d'envoi par API (Brevo, Resend...).	Render gratuit bloque les ports SMTP 25, 465 et 587 : aucun e-mail ne part.
Haute	Corriger SMTP_USER (adresse complète de la boîte) et EMAIL_FROM (adresse @paradigital.tech).	L'identifiant actuel est refusé par LWS.

Priorité	Action	Pourquoi
Moyenne	Stockage durable des CV : disque persistant Render (payant) ou Supabase Storage.	Le disque du plan gratuit est effacé à chaque redéploiement.
Moyenne	Créer un second projet Supabase pour le développement.	Aujourd'hui, les tests en local modifient les données du site en ligne.
Basse	Traiter les vulnérabilités signalées par <code>npm audit</code> .	Hygiène de sécurité.
Basse	Supprimer la demande de contact « TEST — à supprimer » depuis l'administration.	Donnée de test créée pendant la vérification.
Optionnel	Sous-domaine <code>api.paradigital.tech</code> (CNAME vers l'API Render).	Plus professionnel ; permet de changer d'hébergeur d'API sans reconstruire le site.

Aide-mémoire

Élément	Adresse
Site public	<code>https://paradigital.tech</code>
Site (adresse Render)	<code>https://software-house-web-7iah.onrender.com</code>
API (adresse Render)	<code>https://software-house-api-7iah.onrender.com/api/v1</code>
État de l'API et de la base	<code>.../api/v1/health</code>
Espace d'administration	<code>https://paradigital.tech/admin/login</code>
Code source	<code>https://github.com/sete-sarr/software_house</code>
Tableau de bord Render	<code>https://dashboard.render.com</code>
Tableau de bord Supabase	<code>https://supabase.com/dashboard</code>
Zone DNS	Espace client LWS → Domaines → <code>paradigital.tech</code> → Zone DNS
Documentation du projet	<code>DEPLOYMENT.md</code> (option C — Render), <code>ENVIRONMENT.md</code> , <code>render.yaml</code>